

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Khoa Công nghệ thông tin 1



Báo cáo bài tập lớn

Các hệ thống phân tán

**Đề tài: Phát triển hệ thống soạn thảo
văn bản cộng tác**

Giảng viên: Kim Ngọc Bách

Lớp: Các hệ thống phân tán - 03

Sinh viên: Bùi Đức Mạnh - B22DCCN519

Mai Xuân Linh - B22DCCN495

Hà Nội – 2026

I. TỔNG QUAN HỆ THỐNG.....	4
1.1. Giới thiệu.....	4
1.2. Mục tiêu đề tài.....	4
1.3. Công nghệ sử dụng.....	4
2.1. Kiến trúc tổng thể.....	5
2.2. Cấu trúc thư mục dự án.....	5
2.3. Kiến trúc Backend.....	6
2.3.1. Xử lý HTTP (REST API)	7
2.3.2. Xử lý WebSocket (Yjs).....	7
2.4. Kiến trúc Frontend	7
2.4.1. Luồng khởi tạo Editor	7
2.4.2. Phân quyền phía Client	7
2.5. Cơ sở dữ liệu	8
III. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU	8
3.1. Thuật toán CRDT – Yjs	8
3.1.1. Nguyên lý CRDT trong Yjs	8
3.1.2. Cấu trúc dữ liệu dùng trong dự án	9
3.2. Mô hình phòng (Room) trên server.....	9
3.3. Luồng đồng bộ dữ liệu	10
3.3.1. Quy trình Client kết nối mới.....	10
3.3.2. Luồng chỉnh sửa thời gian thực	10
3.3.3. Xử lý xung đột chỉnh sửa đồng thời	11
3.4. Mô hình Awareness (Hiện diện người dùng).....	12
3.5. Cơ chế Auto-save và Quản lý phiên bản.....	12
3.6. Cơ chế Restore phiên bản.....	12
IV. GIAO THỨC TRAO ĐỔI DỮ LIỆU.....	14
4.1. REST API (HTTP).....	14
4.1.1. Xác thực	14
4.1.2. Quản lý tài liệu.....	14
4.1.3. Chia sẻ & Phân quyền.....	14
4.1.4. Phiên bản.....	15
4.2. Giao thức WebSocket (Yjs Protocol).....	15
4.2.1. Handshake và Đồng bộ ban đầu	15
4.2.2. Cấu trúc Message nhị phân.....	16
4.2.3. Kiểm soát luồng và Bảo mật.....	17

4.2.4. Keepalive Ping/Pong	17
4.3. Cơ chế Xác thực	17
V. THỬ NGHIỆM VÀ ĐÁNH GIÁ	18
5.1. Môi trường thử nghiệm	18
5.2. Kịch bản thử nghiệm chức năng	18
5.2.1. Đăng ký và Đăng nhập	18
5.2.2. Quản lý tài liệu.....	18
5.2.3. Cộng tác thời gian thực.....	19
5.2.4. Phân quyền.....	19
5.2.5. Lịch sử phiên bản.....	20
5.3. Đánh giá kết quả.....	20
5.3.1. Những gì đã đạt được	20
5.3.2. Hạn chế và Hướng phát triển	21
5.4. So sánh với yêu cầu đề tài.....	21
VI. KẾT LUẬN	23

I. TỔNG QUAN HỆ THỐNG

1.1. Giới thiệu

Collaborative Editor là một ứng dụng soạn thảo văn bản cộng tác theo thời gian thực, cho phép nhiều người dùng cùng chỉnh sửa một tài liệu đồng thời qua mạng Internet — tương tự Google Docs. Hệ thống được xây dựng theo kiến trúc phân tán client-server, sử dụng thuật toán CRDT (Conflict-free Replicated Data Type) thông qua thư viện Yjs để đảm bảo tính nhất quán dữ liệu mà không cần khóa (lock-free).

1.2. Mục tiêu đề tài

Đề tài hướng đến các mục tiêu kỹ thuật sau:

- Xây dựng hệ thống cho phép nhiều người dùng cùng chỉnh sửa một tài liệu theo thời gian thực qua mạng.
- Áp dụng cơ chế đồng bộ phân tán dựa trên CRDT để tự động giải quyết xung đột chỉnh sửa đồng thời.
- Triển khai giao tiếp hai chiều theo thời gian thực bằng giao thức WebSocket.
- Xây dựng hệ thống phân quyền, quản lý phiên bản và chia sẻ tài liệu.

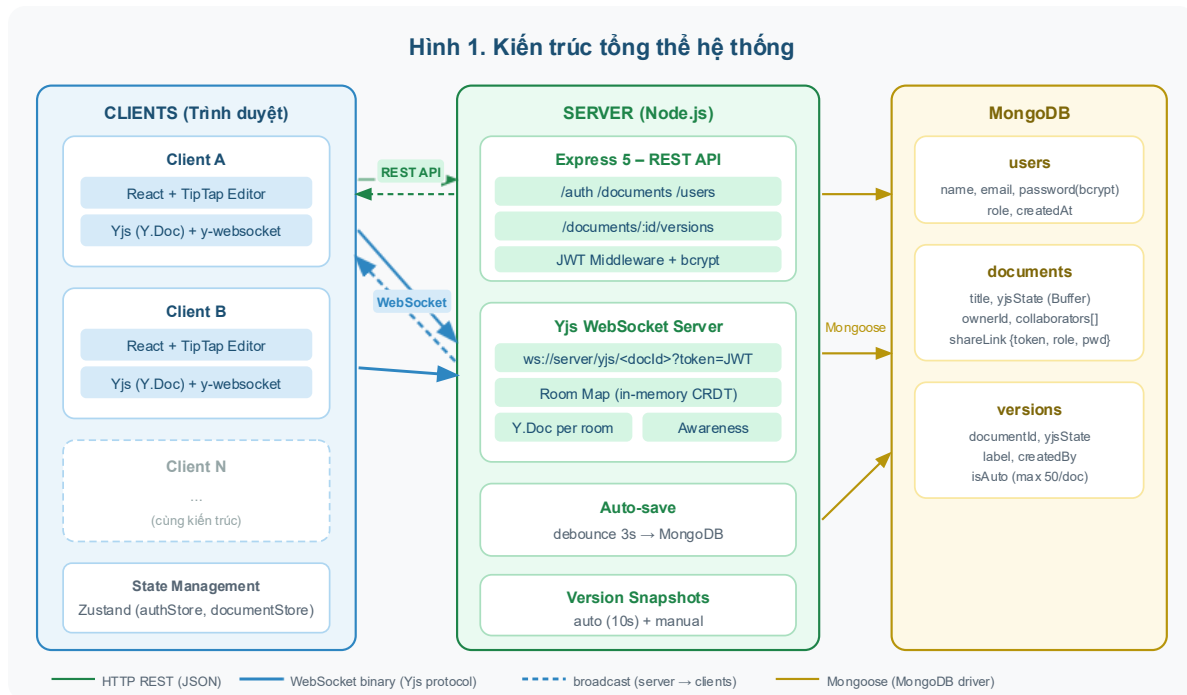
1.3. Công nghệ sử dụng

Thành phần	Công nghệ	Mục đích
Backend	Node.js + Express 5	HTTP server & REST API
	MongoDB + Mongoose	Lưu trữ tài liệu và trạng thái Yjs
	Yjs + y-protocols	CRDT engine đồng bộ thời gian thực
	ws (WebSocket)	Kết nối realtime server-client
	JWT + bcryptjs	Xác thực & mã hóa mật khẩu
Frontend	React 18 + Vite 6	UI framework & build tool
	TipTap v2 (ProseMirror)	Rich text editor
	y-websocket + Yjs	CRDT client-side provider
	Zustand	Quản lý state toàn cục
	Ant Design v6 + Tailwind CSS	UI components & styling

II. KIẾN TRÚC HỆ THỐNG

2.1. Kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình Client-Server hai tầng. Server đóng vai trò trung tâm điều phối và lưu trữ, trong khi mỗi Client (trình duyệt) duy trì một bản sao CRDT cục bộ đồng bộ với server qua WebSocket.



[Sơ đồ kiến trúc tổng thể – Client-Server với WebSocket và REST API]

Hình 1. Kiến trúc tổng thể hệ thống

Hệ thống sử dụng hai kênh giao tiếp song song cho mỗi client:

- REST API (HTTP/HTTPS): dùng cho các thao tác CRUD — đăng nhập, tạo/xóa tài liệu, quản lý phân quyền, lịch sử phiên bản.
- WebSocket (ws://): dùng để truyền realtime các cập nhật CRDT (Yjs update) và thông tin Awareness (con trỏ, trạng thái online).

2.2. Cấu trúc thư mục dự án

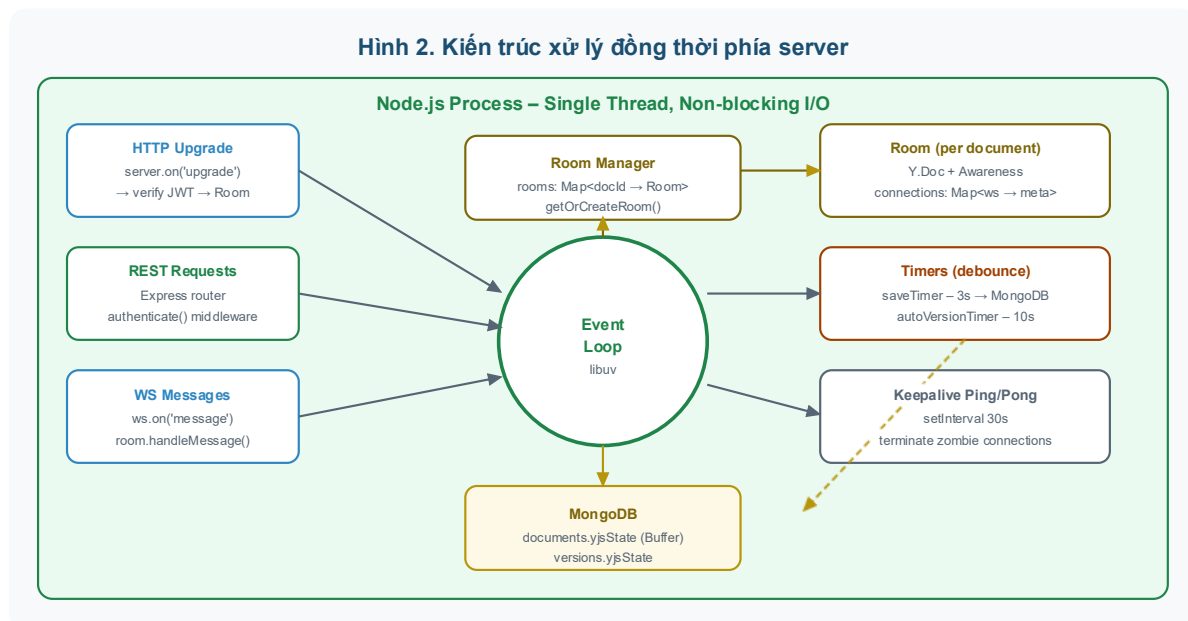
Dự án được tổ chức theo monorepo gồm hai package độc lập:

Đường dẫn	Vai trò
backend/src/index.js	Entry point – khởi tạo Express + HTTP server + đính kèm Yjs WebSocket

Đường dẫn	Vai trò
backend/src/ws/yjsServer.js	Toàn bộ logic WebSocket: Room, đồng bộ CRDT, Awareness, auto-save
backend/src/models/	Mongoose schemas: User, Document (chứa yjsState), Version (snapshot)
backend/src/controllers/	Business logic cho auth, document, user, version
backend/src/middlewares/	JWT authentication middleware, error handler
frontend/src/yjs/	provider.js (WebsocketProvider), awareness.js, yjsConfig.js
frontend/src/hooks/useEditor.js	Orchestrator: khởi tạo Y.Doc, provider, TipTap editor với Collaboration extension
frontend/src/hooks/useSocket.js	Quản lý trạng thái kết nối và danh sách người dùng online
frontend/src/store/	Zustand stores: authStore (JWT token, user), documentStore (danh sách tài liệu)

2.3. Kiến trúc Backend

Backend là một Node.js process duy nhất, nhưng xử lý đồng thời nhiều kết nối nhờ mô hình event-loop non-blocking của Node.js. Server quản lý hai loại kết nối đồng thời:



[Sơ đồ luồng xử lý trên Server – Event Loop xử lý REST và WebSocket song song]

Hình 2. Kiến trúc xử lý đồng thời phía server

2.3.1. Xử lý HTTP (REST API)

Express 5 tiếp nhận các HTTP request thông thường. Mỗi request đi qua middleware authenticate (kiểm tra JWT) trước khi đến controller tương ứng. Tất cả thao tác với MongoDB đều dùng async/await.

2.3.2. Xử lý WebSocket (Yjs)

Khi có request HTTP Upgrade, server kiểm tra URL prefix /yjs/ và xác thực JWT từ query parameter token. Sau đó chuyển giao cho WebSocketServer (thư viện ws). Mỗi document mở sẽ tạo một Room trong bộ nhớ. Room chứa:

- Một Y.Doc riêng – bản sao CRDT của tài liệu trên server.
- Một Awareness – object theo dõi trạng thái hiện diện (presence) của từng client.
- Một Map connections – ánh xạ WebSocket → metadata (userId, role, controlledIds).
- Hai timer: saveTimer (debounce 3s để auto-save yjsState vào MongoDB) và autoVersionTimer (debounce 10s để lưu snapshot phiên bản khi phòng trống).

2.4. Kiến trúc Frontend

Frontend là một Single Page Application (SPA) được xây dựng với React 18 và Vite 6. Trạng thái toàn cục được quản lý bởi Zustand, còn trạng thái CRDT được quản lý bởi Yjs.

2.4.1. Luồng khởi tạo Editor

Khi người dùng mở một tài liệu (EditorPage), hook useCollaborativeEditor thực hiện các bước sau theo thứ tự:

- Tạo một Y.Doc mới (createYDoc).
- Tạo WebSocketProvider kết nối tới `ws://server/yjs/<documentId>?token=<JWT>`.
- Đăng ký sự kiện sync – khi provider phát tín hiệu sync=true, editor được đánh dấu isReady.
- Khởi tạo TipTap editor với hai extension quan trọng: Collaboration (liên kết với Y.Doc) và CollaborationCursor (hiển thị con trỏ của người khác qua Awareness).

2.4.2. Phân quyền phía Client

Vai trò của người dùng (owner / editor / viewer) được lấy từ REST API khi fetch thông tin tài liệu. Dựa vào đó:

- Viewer: editor bị set editable=false, server sẽ drop silently mọi Yjs update message từ kết nối này.
- Editor / Owner: toàn quyền chỉnh sửa.
- Chỉ owner mới thấy nút "Chia sẻ" và có thể quản lý cộng tác viên.

2.5. Cơ sở dữ liệu

MongoDB được dùng để lưu trữ ba loại dữ liệu chính:

Collection	Trường quan trọng	Mô tả
users	name, email, password (bcrypt), role	Thông tin tài khoản người dùng
documents	title, yjsState (Buffer), ownerId, collaborators[], shareLink	Tài liệu và trạng thái CRDT serialized
versions	documentId, yjsState, label, createdBy, isAuto	Snapshot lịch sử phiên bản (tối đa 50/tài liệu)

Trường yjsState trong Document và Version lưu trạng thái CRDT dưới dạng nhị phân (Binary Buffer) — đây là kết quả của Y.encodeStateAsUpdate(ydoc). Nhờ đó, khi client kết nối lại, server có thể khôi phục toàn bộ lịch sử chỉnh sửa.

III. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU

3.1. Thuật toán CRDT – Yjs

Hệ thống sử dụng Yjs — một thư viện CRDT (Conflict-free Replicated Data Type) hiệu năng cao. CRDT là một họ cấu trúc dữ liệu toán học cho phép nhiều bản sao độc lập được cập nhật đồng thời và hội tụ về cùng một trạng thái mà không cần khóa (lock) hay máy chủ tập trung để giải quyết xung đột.

3.1.1. Nguyên lý CRDT trong Yjs

Yjs triển khai thuật toán YATA (Yet Another Transformation Approach), một dạng CRDT theo thứ tự nhân quả (causal ordering). Mỗi thao tác chèn/xóa ký tự được biểu diễn như một Item với:

- id: {client: clientId, clock: lamportClock} – định danh duy nhất toàn cầu.
- origin / rightOrigin: tham chiếu đến Item bên trái và bên phải tại thời điểm chèn.
- content: nội dung thực tế (ký tự, phân tử, v.v.).
- deleted: cờ xóa mềm (tombstone) — Item không bao giờ bị xóa thực sự khỏi cấu trúc.

Nhờ cách thiết kế này, hai thao tác đồng thời từ hai client khác nhau luôn có thể được hợp nhất (merge) theo thứ tự xác định, đảm bảo tính hội tụ (strong eventual consistency).

3.1.2. Cấu trúc dữ liệu dùng trong dự án

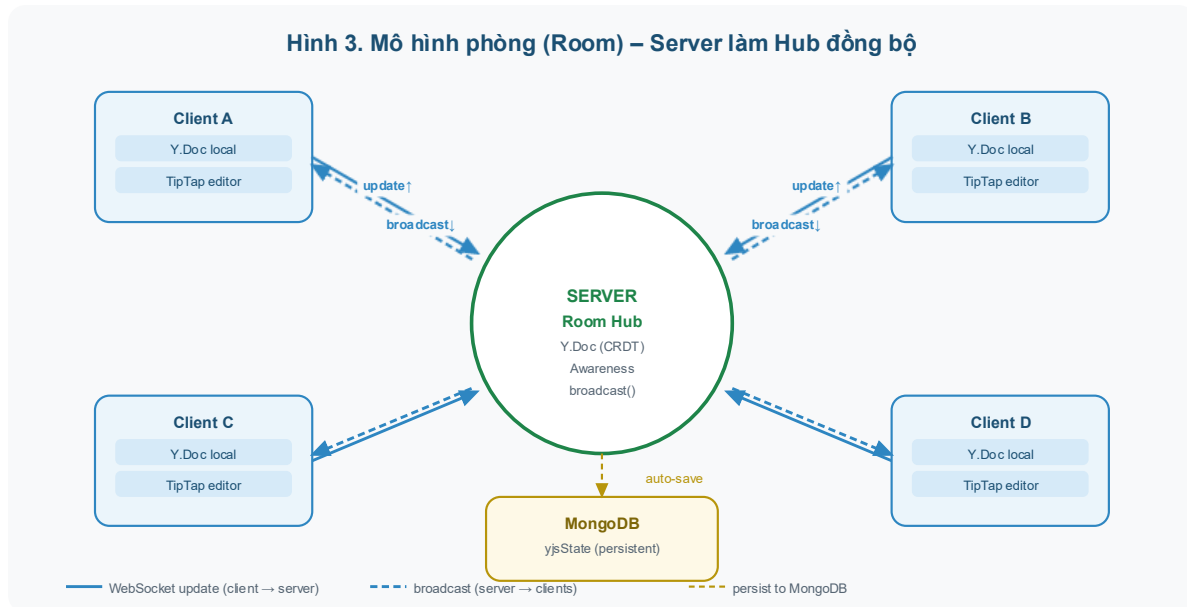
Tài liệu văn bản được lưu trong Y.XmlFragment (thay vì Y.Text thông thường) vì TipTap sử dụng ProseMirror làm engine, cần biểu diễn cấu trúc cây XML (đoạn văn, heading, bảng, v.v.):

```
// yjsConfig.js
const ydoc = new Y.Doc();
const fragment = ydoc.getXmlFragment('default'); // tên field 'default'

// TipTap Collaboration extension liên kết editor với fragment
Collaboration.configure({ document: ydoc })
```

3.2. Mô hình phòng (Room) trên server

Server duy trì một Map toàn cục rooms: documentId → Room. Đây là mô hình hub-and-spoke: mọi update từ client A đều đi qua server trước khi broadcast đến client B.



[Sơ đồ mô hình Room – server là hub trung tâm giữa các clients]

Hình 3. Mô hình phòng (Room) – server làm hub đồng bộ

Khi client đầu tiên kết nối vào một document, Room được tạo và load yjsState từ MongoDB. Khi người dùng cuối cùng rời đi, Room được giữ thêm 10 giây (autoVersionTimer) để lưu snapshot, sau đó bị dọn dẹp khỏi bộ nhớ.

3.3. Luồng đồng bộ dữ liệu

3.3.1. Quy trình Client kết nối mới

Khi một client kết nối vào Room, server thực hiện hai bước khởi tạo ngay lập tức:

1. Gửi Sync Step 1 (SyncStep1): Server gửi vector clock (state vector) của Y.Doc hiện tại. Client dùng thông tin này để biết server đang ở trạng thái nào.
2. Gửi Awareness states hiện tại: Server gửi trạng thái hiện diện (con trỏ, màu sắc) của tất cả client đang online trong phòng để client mới có thể hiển thị ngay.

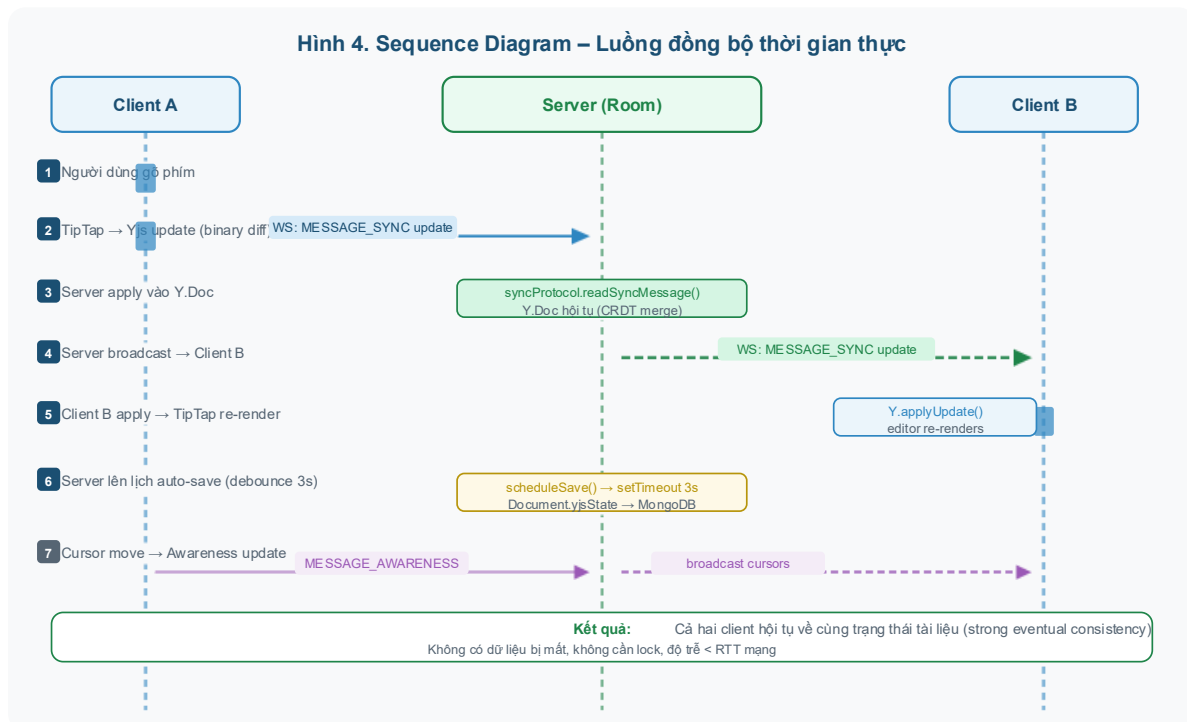
Phía client (y-websocket provider), sau khi nhận SyncStep1, sẽ tự động gửi lại SyncStep2 chứa tất cả update mà client có nhưng server chưa có. Server nhận SyncStep2 và apply vào Y.Doc, sau đó broadcast cho các client còn lại.

3.3.2. Luồng chỉnh sửa thời gian thực

Mỗi khi người dùng gõ phím hoặc định dạng văn bản trong TipTap:

- TipTap (qua ProseMirror) tạo ra một transaction.

- Collaboration extension của TipTap dịch transaction thành một Yjs update (binary diff).
- y-websocket provider gửi update này lên server dưới dạng message nhị phân loại MESSAGE_SYNC.
- Server nhận, apply update vào Y.Doc của Room (gọi syncProtocol.readSyncMessage), sau đó broadcast update đến tất cả connection khác trong Room (trừ người gửi).
- Mỗi client nhận update và apply vào Y.Doc cục bộ của mình. TipTap tự động render lại nội dung.
- Server lên lịch auto-save (debounce 3 giây) — ghi yjsState vào MongoDB.



[Sequence diagram – luồng đồng bộ khi Client A chỉnh sửa đến Server đến Client B]

Hình 4. Sequence diagram luồng đồng bộ chỉnh sửa thời gian thực

3.3.3. Xử lý xung đột chỉnh sửa đồng thời

Giả sử hai client A và B cùng chỉnh sửa tại vị trí tương tự trong khi mạng bị chậm:

- Client A chèn "Hello" tại offset 5, tạo update_A.
- Client B chèn "World" tại offset 5 (chưa nhận update_A), tạo update_B.
- Server nhận update_A trước: apply vào Y.Doc → văn bản trở thành "...Hello..."
- Server nhận update_B: Yjs sử dụng thuật toán YATA để xác định vị trí chèn dựa trên origin (Item tham chiếu) — không phải offset số. Kết quả: "...HelloWorld..." hoặc "...WorldHello..." tùy thuộc vào clientId (tie-breaking). Thứ tự này nhất quán trên tất cả các node.

Không có dữ liệu nào bị mất. Cả hai thao tác đều được bảo toàn và hội tụ về cùng một trạng thái trên tất cả client.

3.4. Mô hình Awareness (Hiện diện người dùng)

Song song với đồng bộ nội dung tài liệu, hệ thống đồng bộ thông tin hiện diện người dùng qua giao thức Awareness của Yjs. Awareness là một CRDT riêng biệt, không persistent (mất khi mạng đứt), dùng để truyền:

- Thông tin người dùng: id, name, color (màu con trỏ riêng mỗi user).
- Vị trí con trỏ (cursor position) và vùng chọn (selection range) trong tài liệu.

Server không diễn giải nội dung Awareness — chỉ broadcast đến các client còn lại. Khi một client ngắt kết nối, server gọi `awarenessProtocol.removeAwarenessStates()` để xóa trạng thái của client đó, đảm bảo con trỏ biến mất khỏi màn hình của người khác.

3.5. Cơ chế Auto-save và Quản lý phiên bản

Cơ chế	Thời gian / Điều kiện	Hành động
Auto-save yjsState	3 giây sau lần chỉnh sửa cuối (debounce)	Ghi <code>Y.encodeStateAsUpdate(ydoc)</code> vào <code>Document.yjsState</code> trong MongoDB
Auto-save version snapshot	10 giây sau khi người dùng cuối rời phòng (debounce)	Tạo Version mới (<code>isAuto=true</code>), rate-limit 2 phút, tối đa 50 versions
Manual snapshot	Theo yêu cầu người dùng	Tạo Version với label tùy chỉnh (<code>isAuto=false</code>)
Keepalive ping/pong	Mỗi 30 giây	Giữ WebSocket sống, terminate nếu không nhận pong

3.6. Cơ chế Restore phiên bản

Quy trình khôi phục phiên bản được thiết kế cẩn thận để tránh race condition giữa WebSocket và REST API:

- API nhận request restore: gọi kickRoom() — đánh dấu room.killed=true, hủy tất cả timer, đóng toàn bộ kết nối WebSocket trong room, xóa room khỏi bộ nhớ.
- Lưu backup tự động của trạng thái hiện tại trước khi ghi đè.
- Ghi yjsState mới (của version cũ) vào MongoDB bằng findByIdAndUpdate.
- Clients bị ngắt kết nối, WebSocketProvider phía client tự động reconnect.
- Khi reconnect, server tạo Room mới và load yjsState mới từ MongoDB.

Bước kick room TRƯỚC khi ghi DB là quan trọng — nếu đảo ngược thứ tự, timer debounce 3 giây của room cũ có thể flush Y.Doc cũ (đang giữ trong RAM) đè lên yjsState mới vừa restore.

IV. GIAO THỨC TRAO ĐỔI DỮ LIỆU

4.1. REST API (HTTP)

REST API sử dụng HTTP/HTTPS làm giao thức vận chuyển, JSON làm định dạng dữ liệu. Xác thực dựa trên JWT Bearer Token trong header Authorization.

4.1.1. Xác thực

Method	Endpoint	Mô tả
POST	/auth/register	Đăng ký tài khoản mới
POST	/auth/login	Đăng nhập, nhận JWT token
GET	/auth/me	Lấy thông tin user hiện tại (cần JWT)

4.1.2. Quản lý tài liệu

Method	Endpoint	Mô tả
POST	/documents	Tạo tài liệu mới
GET	/documents	Danh sách tài liệu của user
GET	/documents/:id	Chi tiết một tài liệu
PATCH	/documents/:id	Cập nhật tiêu đề tài liệu
DELETE	/documents/:id	Xóa tài liệu (chỉ owner)
GET	/documents/:id/state	Lấy Yjs state dạng base64

4.1.3. Chia sẻ & Phân quyền

Method	Endpoint	Mô tả
POST	/documents/:id/share	Mời cộng tác viên qua email
GET	/documents/:id/permissions	Danh sách quyền truy cập
DELETE	/documents/:id/share/:userId	Thu hồi quyền của một user
POST	/documents/:id/share-link	Tạo link chia sẻ (có thể đặt mật khẩu)
DELETE	/documents/:id/share-link	Thu hồi link chia sẻ

Method	Endpoint	Mô tả
POST	/documents/join/:token	Tham gia tài liệu qua link

4.1.4. Phiên bản

Method	Endpoint	Mô tả
GET	/documents/:id/versions	Danh sách phiên bản
POST	/documents/:id/versions	Lưu snapshot thủ công
GET	/documents/:id/versions/:vId/state	Lấy nội dung của một phiên bản
POST	/documents/:id/versions/:vId/restore	Khôi phục về phiên bản cũ

4.2. Giao thức WebSocket (Yjs Protocol)

Kết nối WebSocket được thiết lập tại:

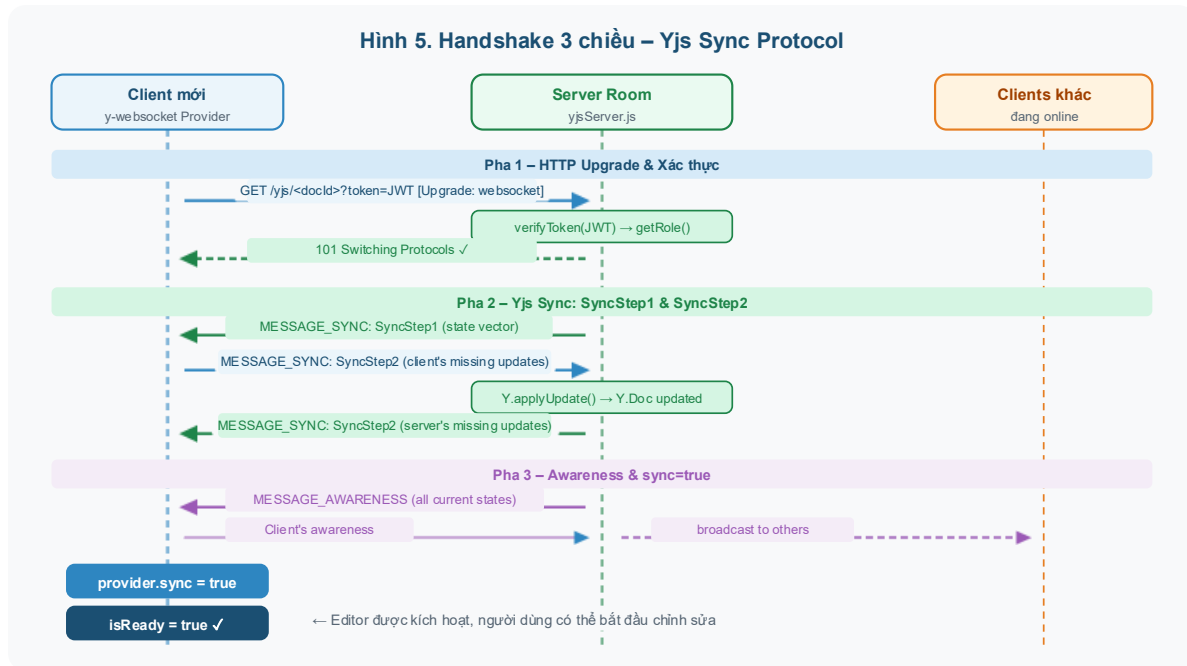
```
ws://<server>/yjs/<documentId>?token=<JWT>
```

Sau khi kết nối, toàn bộ thông điệp được truyền dưới dạng nhị phân (binary). Giao thức Yjs định nghĩa hai loại message:

Type ID	Tên	Mô tả
0	MESSAGE_SYNC	Mang payload Yjs sync protocol: SyncStep1 (trao đổi state vector), SyncStep2 (gửi update còn thiếu), hoặc Update (thay đổi mới). Dữ liệu là binary CRDT diff.
1	MESSAGE_AWARENESS	Mang payload Awareness: thông tin hiện diện người dùng (tên, màu, vị trí con trỏ). Được broadcast đến tất cả client trong Room.

4.2.1. Handshake và Đồng bộ ban đầu

Quy trình bắt tay khi client mới kết nối (giao thức 3 chiều):



[Sequence diagram – Yjs 3-way handshake: SyncStep1 → SyncStep2 → Update broadcast]

Hình 5. Handshake 3 chiều khi client kết nối mới

- Server → Client: SyncStep1 (state vector của server).
- Client → Server: SyncStep2 (tồn bộ update mà client có nhưng server thiếu).
- Server apply SyncStep2 vào Y.Doc, sau đó broadcast Update đến các client khác.
- Server → Client mới: SyncStep2 ngược lại (update mà server có nhưng client chưa có).

Sau bước này, provider phía client phát sự kiện sync=true — TipTap editor được kích hoạt (isReady=true) và người dùng có thể bắt đầu chỉnh sửa.

4.2.2. Cấu trúc Message nhị phân

Mỗi message WebSocket là một mảng byte được encode bằng lib0 (thư viện mã hóa nhị phân hiệu năng cao của Yjs):

[messageType (varint) | payload (binary)]

Với MESSAGE_SYNC (type=0):

[0 (varint) | syncMessageType (varint) | syncPayload (binary)]

Với MESSAGE_AWARENESS (type=1):

[1 (varint) | awarenessUpdate (binary)]

Sử dụng varint (variable-length integer encoding) giúp giảm overhead cho các số nhỏ. Update nhị phân CRDT thường chỉ vài chục byte cho một lần gõ phím, rất hiệu quả về băng thông.

4.2.3. Kiểm soát luồng và Bảo mật

Server kiểm tra role của mỗi connection trước khi xử lý message:

- viewer: Các SyncStep2 và Update message bị drop silently — người xem không thể ghi dữ liệu vào Y.Doc ngay cả khi họ tự gửi WebSocket message.
- editor / owner: Xử lý đầy đủ cả hai loại message.

Việc kiểm soát ở tầng WebSocket bổ sung cho kiểm soát ở tầng UI (editor.editable=false), tạo thành bảo vệ hai lớp (defense in depth).

4.2.4. Keepalive Ping/Pong

Server gửi WebSocket ping mỗi 30 giây. Nếu không nhận pong trong chu kỳ tiếp theo, connection bị terminate. Cơ chế này phát hiện và dọn dẹp các kết nối zombie (client đứt mạng đột ngột không gửi Close frame).

4.3. Cơ chế Xác thực

Xác thực sử dụng JWT (JSON Web Token) với algorithm HS256. Token có thời hạn 7 ngày (cấu hình qua JWT_EXPIRES_IN). Mật khẩu được hash bằng bcrypt với salt rounds = 12.

Luồng	Cơ chế
REST API	Header: Authorization: Bearer <JWT>. Middleware authenticate() verify token và attach user vào req.user.
WebSocket	Query param: ?token=<JWT>. Kiểm tra trong HTTP Upgrade event trước khi nâng cấp kết nối. Không hợp lệ → trả 401 và destroy socket.
Share link	Token ngẫu nhiên UUID4 lưu trong document.shareLink.token. Mật khẩu share link được hash bằng bcrypt riêng.

V. THỬ NGHIỆM VÀ ĐÁNH GIÁ

5.1. Môi trường thử nghiệm

Thành phần	Cấu hình
Máy chủ	Laptop cá nhân
Hệ điều hành	Window
Node.js	v18+ (ES Modules)
MongoDB	MongoDB Atlas
Trình duyệt client	Google Chrome
Mạng	Localhost

5.2. Kịch bản thử nghiệm chức năng

5.2.1. Đăng ký và Đăng nhập

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC01	Đăng ký tài khoản mới với email hợp lệ	Tạo user thành công, HTTP 201	Đạt
TC02	Đăng nhập đúng email/mật khẩu	Trả JWT token, HTTP 200	Đạt
TC03	Đăng nhập sai mật khẩu	HTTP 401, thông báo lỗi	Đạt
TC04	Truy cập API không có JWT	HTTP 401	Đạt

5.2.2. Quản lý tài liệu

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC05	Tạo tài liệu mới	Document tạo thành công, xuất hiện trên Dashboard	Đạt
TC06	Đổi tên tài liệu	Tiêu đề cập nhật real-time trên tất cả client	Đạt

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC07	Xóa tài liệu (owner)	Tài liệu bị xóa, biến mất khỏi danh sách	Đạt
TC08	Xóa tài liệu (non-owner)	HTTP 404 (không trả 403 để tránh lộ thông tin)	Đạt

5.2.3. Cộng tác thời gian thực

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC09	2 client cùng mở một tài liệu	Cả hai thấy cùng nội dung sau sync	Đạt
TC10	Client A gõ văn bản	Client B thấy text xuất hiện < 200ms	Đạt
TC11	Hai client gõ đồng thời cùng vị trí	Cả hai update được merge, không mất dữ liệu, hội tụ nhất quán	Đạt
TC12	Hiển thị con trỏ người dùng khác	Con trỏ màu sắc riêng, tên hiển thị, di chuyển theo thời gian thực	Đạt
TC13	Client ngắt kết nối	Con trỏ biến mất, avatar nhạt dần trong sidebar	Đạt
TC14	Client reconnect sau khi mất mạng	Đồng bộ lại đầy đủ, không mất nội dung chỉnh sửa offline	Đạt

5.2.4. Phân quyền

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC15	Viewer cố gõ vào editor	Editor bị khóa (editable=false), UI không cho phép	Đạt
TC16	Viewer gửi Yjs update qua WebSocket thủ công	Server drop silently, không apply vào Y.Doc	Đạt

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC17	Owner mời user qua email, cấp role editor	User xuất hiện trong danh sách permissions, có thể chỉnh sửa	Đạt
TC18	Tham gia qua share link có mật khẩu	Yêu cầu nhập mật khẩu, sai → lỗi, đúng → vào được	Đạt

5.2.5. Lịch sử phiên bản

TC	Mô tả	Kết quả mong đợi	Kết quả thực tế
TC19	Lưu snapshot thủ công với label	Version xuất hiện trong lịch sử với label đúng	Đạt
TC20	Auto-save version khi người cuối rời phòng	Sau 10s, version tự động tạo (isAuto=true)	Đạt
TC21	Restore về phiên bản cũ	Tài liệu về đúng nội dung, backup tự động tạo trước restore	Đạt
TC22	Restore trong khi có người đang chỉnh sửa	Toàn bộ client bị kick, reconnect về trạng thái mới	Đạt

5.3. Đánh giá kết quả

5.3.1. Những gì đã đạt được

- Đồng bộ thời gian thực hoạt động chính xác — nhiều người dùng chỉnh sửa đồng thời, không mất dữ liệu, hội tụ nhất quán nhờ thuật toán CRDT (Yjs/YATA).
- Hiện thị con trỏ và trạng thái hiện diện (Awareness) của người dùng khác theo thời gian thực.
- Hệ thống phân quyền ba cấp (owner/editor/viewer) được kiểm soát ở cả tầng UI và tầng WebSocket.
- Auto-save thông minh với debounce, tránh ghi quá nhiều vào MongoDB khi người dùng gõ liên tục.
- Lịch sử phiên bản tự động và thủ công, có thể restore về bất kỳ điểm nào với cơ chế an toàn (kick room trước khi ghi).

- Hỗ trợ rich text đầy đủ: heading, table, image, link, code block, task list, tìm kiếm & thay thế.
- Chia sẻ qua link với tùy chọn mật khẩu bảo vệ.
- Xuất tài liệu ra PDF, HTML, Markdown, plain text hoàn toàn phía client.

5.3.2. Hạn chế và Hướng phát triển

- Server đơn (single node): Hệ thống hiện chạy trên một process Node.js duy nhất. Để mở rộng theo chiều ngang (horizontal scaling), cần thay thế in-memory Room Map bằng Redis Pub/Sub hoặc một giải pháp distributed shared state.
- Chưa có rich conflict resolution UI: Người dùng không thấy rõ các xung đột đã được giải quyết như thế nào — phù hợp để bổ sung trong giai đoạn nâng cao.
- Chưa hỗ trợ offline editing đầy đủ: Tuy có auto-reconnect và banner cảnh báo offline, nhưng chưa có IndexedDB persistence (y-indexeddb) để lưu Y.Doc cục bộ qua refresh tab.
- Chưa có undo/redo cộng tác: Extension Collaboration của TipTap đã disable History mặc định để tránh xung đột với CRDT — undo/redo trong môi trường cộng tác cần triển khai riêng với y-undo-manager.
- Giới hạn phiên bản: Tối đa 50 version/tài liệu — cần chiến lược tiered storage (nén version cũ) cho môi trường production.

5.4. So sánh với yêu cầu đề tài

Yêu cầu đề tài	Trạng thái	Ghi chú
Tạo và quản lý tài liệu	Đạt	CRUD đầy đủ
Chỉnh sửa cộng tác thời gian thực	Đạt	Yjs CRDT + WebSocket
Đồng bộ thay đổi, đảm bảo nhất quán	Đạt	Strong eventual consistency
Hiển thị danh sách người dùng đang chỉnh sửa	Đạt	Avatar group + Sidebar
Hiển thị con trỏ người dùng	Đạt	CollaborationCursor + Awareness
Lưu trữ tài liệu trên server	Đạt	MongoDB, auto-save 3s debounce
Giao tiếp mạng WebSocket	Đạt	ws library + HTTP Upgrade

Yêu cầu đề tài	Trạng thái	Ghi chú
Xử lý nhiều client đồng thời	Đạt	Node.js event loop + Room map
Xử lý xung đột chỉnh sửa đồng thời	Đạt	CRDT YATA algorithm
Lịch sử chỉnh sửa và versioning	Đạt (+)	Auto + manual snapshot
Hiển thị con trỏ và vùng chọn người dùng khác	Đạt (+)	CollaborationCursor extension
Phân quyền người dùng (owner/editor/viewer)	Đạt (+)	3-tier RBAC
Undo/redo trong môi trường cộng tác	Chưa đủ	Cần y-undo-manager
Offline editing và đồng bộ khi kết nối lại	Một phần (+)	Auto-reconnect có, chưa có y-indexeddb

VI. KẾT LUẬN

Đồ án đã xây dựng thành công một hệ thống soạn thảo văn bản cộng tác thời gian thực đáp ứng đầy đủ các yêu cầu bắt buộc và hầu hết yêu cầu nâng cao của đề tài. Hệ thống thể hiện rõ các đặc trưng của hệ thống phân tán: giao tiếp mạng qua WebSocket, xử lý đồng thời nhiều client, đồng bộ trạng thái phân tán bằng CRDT, và quản lý tính nhất quán dữ liệu không cần khóa tập trung.

Điểm nổi bật về mặt kỹ thuật là việc tích hợp chặt chẽ giữa ba tầng: thuật toán CRDT (Yjs/YATA) ở tầng dữ liệu, giao thức nhị phân hiệu năng cao ở tầng truyền thông, và rich text editor chuyên nghiệp (TipTap/ProseMirror) ở tầng giao diện. Cơ chế phân quyền hai lớp (UI + WebSocket) và quy trình restore phiên bản an toàn là những thiết kế đặc biệt đáng chú ý.

Hướng phát triển tiếp theo bao gồm: hỗ trợ undo/redo cộng tác bằng y-undo-manager, persistence offline bằng y-indexeddb, và horizontal scaling bằng Redis Pub/Sub để đưa hệ thống vào môi trường production thực tế.